

Structures de données usuelles

Partie I

1. Notions fondamentales

Definition 1.1.1 (Structure de données)

Une structure de données est une manière de représenter les données pour les traiter plus facilement.

1.1 Définitions

Definition 1.1.1 (Structure de données)

Une structure de données est une manière de représenter les données pour les traiter plus facilement.

Les structures de données usuelles que nous verrons :

- Les listes et tableaux (*arrays*)
- Les ensembles (*sets*) et les dictionnaires (*maps*)
- Les piles (*stacks*) et les files (*queues*)

Chacunes de ces structures présentent des situations où il est intéressant de les utiliser.

Definition 1.2.1 (Structure)

Une **collection** est un regroupement d'un ensemble de données.

Une **séquence** est une collection indexée de données, avec un ordre défini.

Definition 1.2.1 (Structure)

Une **collection** est un regroupement d'un ensemble de données.

Une **séquence** est une collection indexée de données, avec un ordre défini.

Definition 1.2.2 (Mutable et Immuable)

Une collection de données est **mutable** si et seulement si elle est modifiable après sa création. Dans le cas contraire nous retrouvons la définition d'une collection **immuable**.

2. Les listes

2.1 Définition

Definition 2.1.1 (Liste)

Une **liste** est une séquence finie de données.

Il existe plein d'implémentations différentes pour cette structure, mais nous nous concentrerons sur les **tableaux**.

2.1 Définition

Definition 2.1.1 (Liste)

Une **liste** est une séquence finie de données.

Il existe plein d'implémentations différentes pour cette structure, mais nous nous concentrerons sur les **tableaux**.

Definition 2.1.2 (Tableau)

Un **tableau** (en anglais “*array*”) est une **liste** dont les éléments sont contigus en mémoire.

Note

L'indexage des éléments commence à 0.

La structure est présente dans la majorité des langages de programmation :

- En python avec *list*
- En Java avec *ArrayList*
- En C++ avec *vector*

2.2 Opérations par défaut

① Syntaxe

On écrira un tableau de la manière suivante : $[1, 2, 3, 5, 7]$.

2.2 Opérations par défaut

① Syntaxe

On écrira un tableau de la manière suivante : $[1, 2, 3, 5, 7]$.

Voici une liste des opérations par défaut pour les **tableaux**:

Opération	Complexité
Accès et modification du k -ème élément	$\mathcal{O}(1)$
Ajout d'un élément à la fin	$\mathcal{O}(\tilde{1})$
Ajout d'un élément au début	$\mathcal{O}(n)$
Insertion d'un élément à l'indice k	$\mathcal{O}(k)$
Recherche d'un élément	$\mathcal{O}(n)$

2.3 Vocabulaire particulier

Soit A et B des listes :

Definition 2.3.1 (Sous-liste)

A est une **sous-liste** (en anglais “*subarray*”) de B si et seulement si elle peut être obtenu en prenant dans l’ordre, les éléments dans B de l’indice i à l’indice j . On écrira $A = B[i; j]$.

2.3 Vocabulaire particulier

Soit A et B des listes :

Definition 2.3.1 (Sous-liste)

A est une **sous-liste** (en anglais “*subarray*”) de B si et seulement si elle peut être obtenu en prenant dans l’ordre, les éléments dans B de l’indice i à l’indice j . On écrira $A = B[i; j]$.

Definition 2.3.2 (Sous-séquence)

A est une **sous-séquence** (en anglais “*subsequence*”) de B si et seulement si elle peut être obtenu par la suppression d’éléments à certains indices de B .

2.3 Vocabulaire particulier

Soit A et B des listes :

Definition 2.3.1 (Sous-liste)

A est une **sous-liste** (en anglais “*subarray*”) de B si et seulement si elle peut être obtenu en prenant dans l’ordre, les éléments dans B de l’indice i à l’indice j . On écrira $A = B[i; j]$.

Definition 2.3.2 (Sous-séquence)

A est une **sous-séquence** (en anglais “*subsequence*”) de B si et seulement si elle peut être obtenu par la suppression d’éléments à certains indices de B .

Definition 2.3.3 (Préfixe et Suffixe)

Un **préfixe** de A est une sous-liste de A qui commence au premier élément.

Un **suffixe** de A est une sous-liste de A qui termine au dernier élément.

2.4 Exercice

Exercise. Soit A un tableau d'entiers de taille n , on souhaite répondre à q requêtes successives qui demandent la somme des éléments de $A[i; j]$, avec $0 \leq i \leq j < n$. Montrer qu'on peut résoudre le problème avec une complexité $\mathcal{O}(q + n)$.

2.5 Solution 1

Naïvement, si nous cherchons à calculer la somme des éléments en itérant sur les valeurs de l'indice i à j , dans le pire des cas, chaque requête serait en $\mathcal{O}(n)$.

Pourquoi donc ? Car chaque requête pourrait demander la somme de $A[0; n]$.

2.5 Solution 1

Naïvement, si nous cherchons à calculer la somme des éléments en itérant sur les valeurs de l'indice i à j , dans le pire des cas, chaque requête serait en $\mathcal{O}(n)$.

Pourquoi donc ? Car chaque requête pourrait demander la somme de $A[0; n]$.

Ce qui amène notre algorithme à avoir une complexité en $\mathcal{O}(qn)$.

2.6 Solution 2

Une technique assez commune, c'est de précalculer des valeurs pour retrouver la somme de $A[i; j]$ en $\mathcal{O}(1)$.

2.6 Solution 2

Une technique assez commune, c'est de précalculer des valeurs pour retrouver la somme de $A[i; j]$ en $\mathcal{O}(1)$.

Il suffit de créer une liste auxiliaire de taille $n + 1$, et qu'on nomme B . A chaque indice i , B stocke la somme de $A[0; i]$.

2.6 Solution 2

Une technique assez commune, c'est de précalculer des valeurs pour retrouver la somme de $A[i; j]$ en $\mathcal{O}(1)$.

Il suffit de créer une liste auxiliaire de taille $n + 1$, et qu'on nomme B . A chaque indice i , B stocke la somme de $A[0; i]$.

Il se trouve qu'on peut générer cette liste en $\mathcal{O}(n)$, car on a :

$$B[0] = 0, \text{ et } \forall i \in \llbracket 1, n \rrbracket, B[i] = B[i - 1] + A[i]$$

2.6 Solution 2

Une technique assez commune, c'est de précalculer des valeurs pour retrouver la somme de $A[i; j]$ en $\mathcal{O}(1)$.

Il suffit de créer une liste auxiliaire de taille $n + 1$, et qu'on nomme B . A chaque indice i , B stocke la somme de $A[0; i]$.

Il se trouve qu'on peut générer cette liste en $\mathcal{O}(n)$, car on a :

$$B[0] = 0, \text{ et } \forall i \in \llbracket 1, n \rrbracket, B[i] = B[i - 1] + A[i]$$

Ainsi, pour retrouver la somme $A[i; j]$, il suffit de constater que :

$$A[i; j] = B[j] - B[i]$$

2.6 Solution 2

Une technique assez commune, c'est de précalculer des valeurs pour retrouver la somme de $A[i; j]$ en $\mathcal{O}(1)$.

Il suffit de créer une liste auxiliaire de taille $n + 1$, et qu'on nomme B . A chaque indice i , B stocke la somme de $A[0; i]$.

Il se trouve qu'on peut générer cette liste en $\mathcal{O}(n)$, car on a :

$$B[0] = 0, \text{ et } \forall i \in \llbracket 1, n \rrbracket, B[i] = B[i - 1] + A[i]$$

Ainsi, pour retrouver la somme $A[i; j]$, il suffit de constater que :

$$A[i; j] = B[j] - B[i]$$

Nous tombons alors sur une complexité $\mathcal{O}(n + q)$, avec n pour la génération de B .

On dit que B est la liste des sommes préfixes de A .

3. Crédits

Auteur et images : Bilel MAHDJOUR
Pour le **Bureau des technologies** de
Polytech Nice Sophia

4. Sources

- USACO guide - Introduction to data structures
- CP algorithms